

# Demo teori Docker minggu 5 - cukup browser

**Bisa dijalankan sepenuhnya melalui browser tanpa memasang Docker, Python, Node.js, Git, atau VS Code pada laptop.** Docker tetap dipasang dan berjalan pada mesin Linux cloud milik Codespaces; browser hanya menjadi editor, terminal, dan jendela aplikasi. Diperlukan akun GitHub, internet, serta kuota Codespaces yang tersedia.

Demo ini menghubungkan konsep teori dengan aplikasi Cloud Notes pada Lab 05. Kegiatan ini formatif dan **tidak menjadi tugas atau penilaian lab terpisah**. Proyek kelompok tetap dikerjakan oleh 3 mahasiswa, dengan presentasi pada minggu 7 dan 14 sesuai panduan proyek.

Bukti di bawah diambil dari Chrome pada **8 Oktober 2026**, setelah membuat Codespace 2 core dari repo ini. Semua tahap build, run, HTTP, browser, restart, dan cleanup benar-benar dijalankan. Angka versi Docker, digest, ID catatan, serta nama Codespace dapat berbeda saat diulang.

## 1. Cerita pembuka: resep, paket makanan, dan dapur

Bayangkan tim membuka beberapa cabang restoran. Mereka ingin setiap cabang menjalankan cara memasak yang sama.

| Istilah Docker                 | Perumpamaan                                          | Hal yang benar secara teknis                                                                      |
|--------------------------------|------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| <code>Dockerfile</code>        | Resep: bahan, urutan kerja, cara menyajikan          | Instruksi untuk membangun image. Bukan aplikasi yang sedang berjalan.                             |
| Build context                  | Bahan yang tersedia untuk resep                      | File yang boleh digunakan build; <code>.dockerignore</code> menyaring file yang tidak diperlukan. |
| Image                          | Paket bahan dan alat yang sudah disiapkan            | Artefak aplikasi beserta dependensi dan metadata; dapat dipakai membuat beberapa container.       |
| Container                      | Satu dapur yang sedang mengerjakan paket tersebut    | Proses terisolasi dari image, dengan lapisan tulisnya sendiri; berbagi kernel host.               |
| Registry                       | Gudang paket yang dapat diambil cabang lain          | Tempat menyimpan dan mengambil image, misalnya Docker Hub.                                        |
| <code>-p 8000:8000</code>      | Nomor pintu pelayanan di luar → nomor pintu di dalam | Port mesin tempat Docker berjalan → port aplikasi di container. Angkanya boleh berbeda.           |
| <code>APP_ENV=classroom</code> | Instruksi operasional untuk cabang kelas             | Environment variable yang diberikan saat container dijalankan; bukan secret pada demo ini.        |
| Multi-stage build              | Dapur persiapan dipisahkan dari paket yang dikirim   | Hasil dari builder dipilih untuk runtime; alat build tidak perlu ikut semuanya.                   |

**Batas perumpamaan:** image bukan mesin virtual dengan kernel sendiri. Resep saja juga tidak menjamin hasil byte-identik apabila base image, dependensi, platform, atau konteks build berubah. Tag image dapat dipindah; digest mengidentifikasi konten image.

Pertanyaan pembuka: “Kalau resep tidak berubah, apakah dapur otomatis sudah berjalan?” Jawaban: belum; `docker build` membuat image, sedangkan `docker run` membuat dan menjalankan container.

## 2. Buka lingkungan online

1. Buka [repo meet5CloudService](#) dan login ke GitHub.
2. Pastikan branch **main** dipilih.
3. Klik tombol **Code** → tab **Codespaces** → **Create codespace on main**. Pilih mesin **2 core** jika opsi ukuran mesin ditampilkan. Untuk memilihnya lebih awal, buka menu tiga titik di tab Codespaces → **New with options**, pilih branch main dan tipe 2 core.
4. Tunggu sampai editor VS Code di browser dan terminal siap. Inisialisasi pertama memerlukan waktu karena image lingkungan dan fitur Docker diambil dari internet.
5. Bila terminal belum tampak, pilih menu **Terminal** → **New Terminal**. Terminal ini adalah Linux di cloud.
6. Jalankan pemeriksaan berikut dari folder repo:

## Gambar 1. Pilih branch main, konfigurasi Docker, dan mesin 2 core

Code → Codespaces → New with options → Create codespace

Codespaces

Q Type 7 to search

Create codespace for  
SeedFlora/meet5CloudService

Branch

This branch will be checked out on creation

 main

Dev container configuration

Your codespace will use this configuration

Cloud Services — Docker Theory Demo

Region

Your codespace will run in the selected region

Southeast Asia

Machine type

Resources for your codespace

2-core

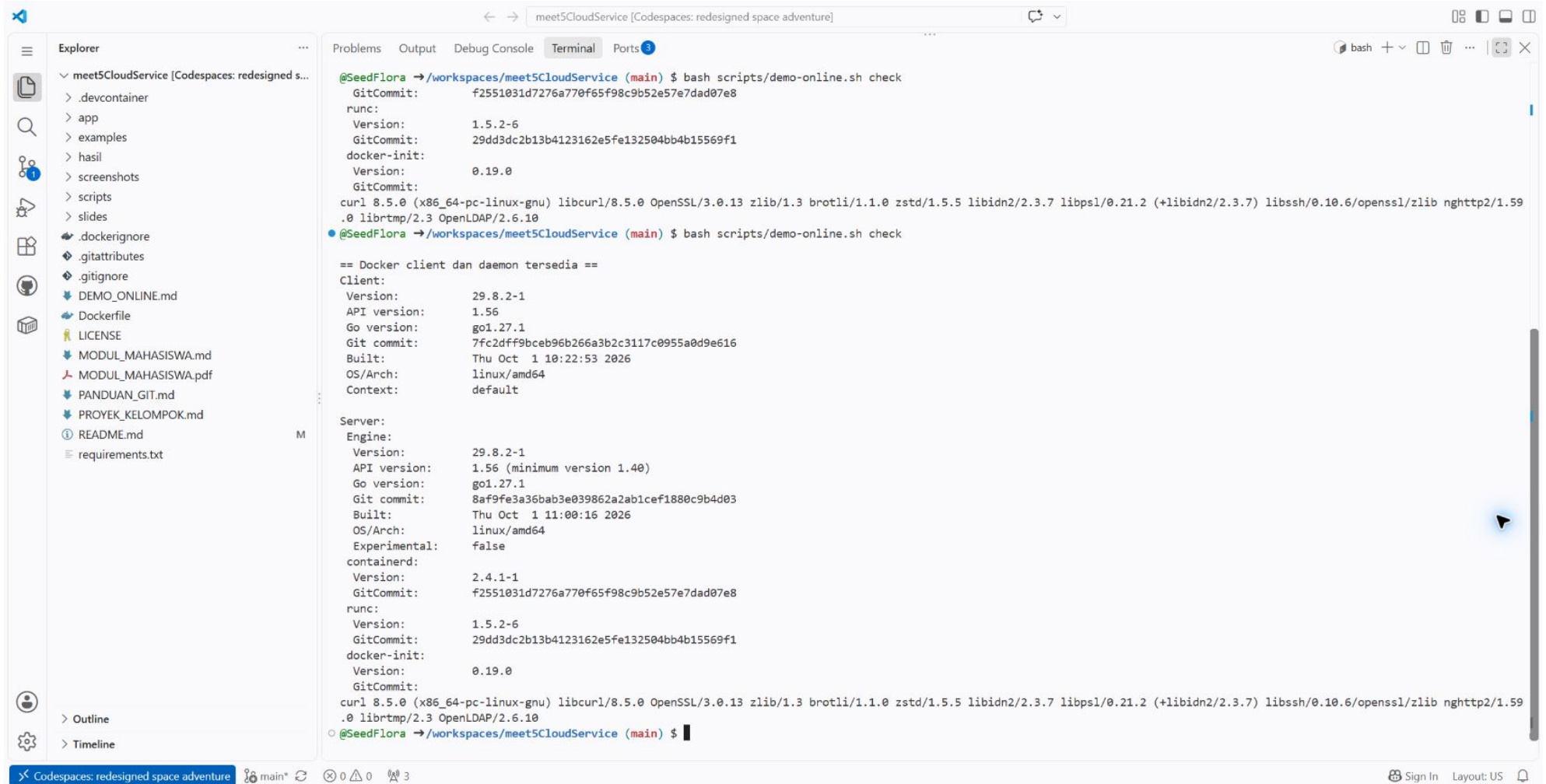
Create codespace

 © 2026 GitHub, Inc. [Terms](#) [Privacy](#) [Security](#) [Status](#) [Community](#) [Docs](#) [Contact](#) [Manage cookies](#) [Do not share my personal information](#)

Ini membuat mesin Linux cloud dari konfigurasi repo. Tidak memasang Docker pada laptop.

## Gambar 2. Docker client dan daemon tersedia

```
bash scripts/demo-online.sh check
```



The screenshot shows a VS Code editor window titled "meet5CloudService [Codespaces: redesigned space adventure]". The Explorer sidebar on the left shows a file tree for the workspace, including files like .devcontainer, app, examples, hasil, screenshots, scripts, slides, .dockerignore, .gitattributes, .gitignore, DEMO\_ONLINE.md, Dockerfile, LICENSE, MODUL\_MAHASISWA.md, MODUL\_MAHASISWA.pdf, PANDUAN\_GIT.md, PROYEK\_KELOMPOK.md, README.md, and requirements.txt. The Terminal panel on the right shows the output of the command `bash scripts/demo-online.sh check`. The output displays the Docker client and server versions, along with the containerd version. The client version is 29.8.2-1, the API version is 1.56, and the Go version is go1.27.1. The server version is also 29.8.2-1, with the same API and Go versions. The containerd version is 2.4.1-1. The output also shows the Docker engine version (0.19.0) and the Docker commit hash (29dd3dc2b13b4123162e5fe132504bb4b15569f1). The output concludes with the message "Docker client dan daemon tersedia ==".

```
@SeedFlora →/workspaces/meet5CloudService (main) $ bash scripts/demo-online.sh check
GitCommit: f2551031d7276a770f65f98c9b52e57e7dad07e8
runc:
Version: 1.5.2-6
GitCommit: 29dd3dc2b13b4123162e5fe132504bb4b15569f1
docker-init:
Version: 0.19.0
GitCommit:
curl 8.5.0 (x86_64-pc-linux-gnu) libcurl/8.5.0 OpenSSL/3.0.13 zlib/1.3 brotli/1.1.0 zstd/1.5.5 libidn2/2.3.7 libpsl/0.21.2 (+libidn2/2.3.7) libssh/0.10.6/openssl/zlib nghttp2/1.59
.0 librtmp/2.3 OpenLDAP/2.6.10
@SeedFlora →/workspaces/meet5CloudService (main) $ bash scripts/demo-online.sh check

== Docker client dan daemon tersedia ==
Client:
Version: 29.8.2-1
API version: 1.56
Go version: go1.27.1
Git commit: 7fc2dfff9bceb96b266a3b2c3117c0955a0d9e616
Built: Thu Oct 1 10:22:53 2026
OS/Arch: linux/amd64
Context: default

Server:
Engine:
Version: 29.8.2-1
API version: 1.56 (minimum version 1.40)
Go version: go1.27.1
Git commit: 8af9fe3a36bab3e039862a2ab1cef1880c9b4d03
Built: Thu Oct 1 11:00:16 2026
OS/Arch: linux/amd64
Experimental: false
containerd:
Version: 2.4.1-1
GitCommit: f2551031d7276a770f65f98c9b52e57e7dad07e8
runc:
Version: 1.5.2-6
GitCommit: 29dd3dc2b13b4123162e5fe132504bb4b15569f1
docker-init:
Version: 0.19.0
GitCommit:
curl 8.5.0 (x86_64-pc-linux-gnu) libcurl/8.5.0 OpenSSL/3.0.13 zlib/1.3 brotli/1.1.0 zstd/1.5.5 libidn2/2.3.7 libpsl/0.21.2 (+libidn2/2.3.7) libssh/0.10.6/openssl/zlib nghttp2/1.59
.0 librtmp/2.3 OpenLDAP/2.6.10
@SeedFlora →/workspaces/meet5CloudService (main) $
```

Client mengirim command; Server/daemon membangun image dan menjalankan container. Versi berbeda masih wajar; yang penting keduanya siap.

**Yang harus terlihat:** bagian `Client` dan `Server` pada `docker version`, kemudian versi `curl`. Jika `Client` muncul tetapi `Server` gagal, daemon belum siap; tunggu inialisasi dan jalankan pemeriksaan lagi.

Konfigurasi `.devcontainer/devcontainer.json` menyiapkan Docker melalui fitur Docker-in-Docker. Aplikasi kelas **tidak otomatis dibuild atau dijalankan** saat workspace dibuka, sehingga dosen dapat memperlihatkan proses build secara bertahap.

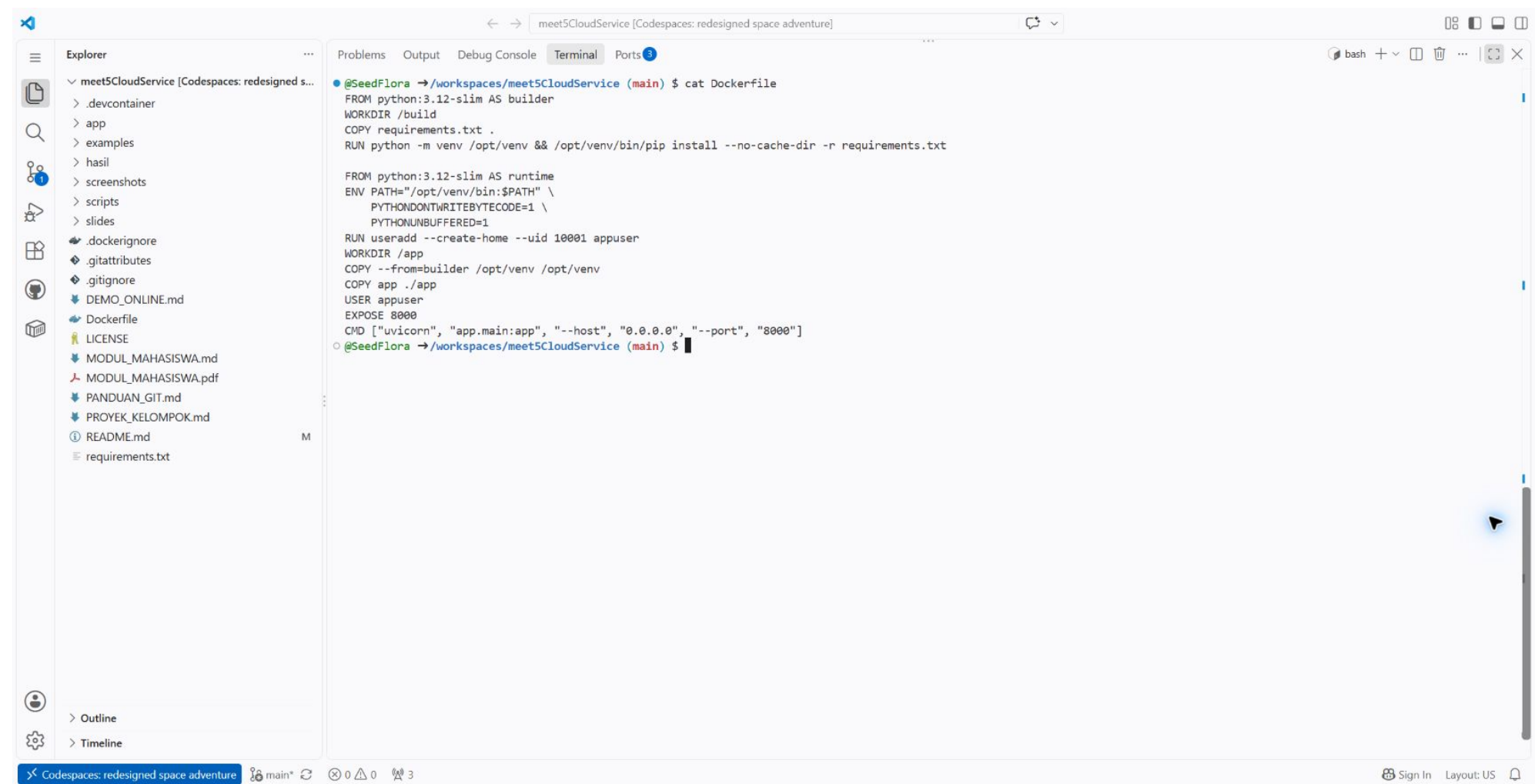
**Jangan tertukar dengan `github.dev`:** menekan titik `.` di halaman GitHub membuka editor ringan; editor itu tidak menyediakan mesin compute dan terminal untuk menjalankan Docker. Gunakan **Codespaces**, yang alamat editornya dapat berakhiran `github.dev` tetapi memiliki nama Codespaces dan terminal Linux.

### 3. Lihat resep, lalu build image

Tampilkan resep asli Lab 05:

## Gambar 3. Resep aplikasi pada repo

```
cat Dockerfile
```



```
@SeedFlora → /workspaces/meet5CloudService (main) $ cat Dockerfile
FROM python:3.12-slim AS builder
WORKDIR /build
COPY requirements.txt .
RUN python -m venv /opt/venv && /opt/venv/bin/pip install --no-cache-dir -r requirements.txt

FROM python:3.12-slim AS runtime
ENV PATH="/opt/venv/bin:$PATH" \
    PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1
RUN useradd --create-home --uid 10001 appuser
WORKDIR /app
COPY --from=builder /opt/venv /opt/venv
COPY app ./app
USER appuser
EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
@SeedFlora → /workspaces/meet5CloudService (main) $
```

Membaca file tanpa mengubahnya. Cocokkan FROM, RUN, COPY, USER, EXPOSE, dan CMD dengan penjelasan di bawah.

Penjelasan yang ditunjukkan sambil membaca file:

- `FROM ... AS builder`: pilih lingkungan Python untuk tahap persiapan.
- `WORKDIR /build`: tentukan direktori kerja, tanpa mengandalkan `RUN cd` pada instruksi sebelumnya.
- `COPY requirements.txt .`: salin daftar dependensi lebih dahulu agar cache instalasi dapat digunakan ketika kode aplikasi saja berubah.
- `RUN ...`: buat virtual environment dan pasang dependensi saat **build**.
- `FROM ... AS runtime`: mulai tahap image runtime baru.
- `COPY --from=builder ...`: ambil virtual environment hasil builder ke runtime.
- `COPY app ./app`: masukkan kode API.
- `USER appuser`: jalankan aplikasi dengan user bukan root.
- `EXPOSE 8000`: dokumentasikan port aplikasi; instruksi ini **tidak** memublikasikan port.
- `CMD [...]`: perintah default ketika container mulai berjalan. Perintah ini tidak dieksekusi ketika build.

Build image:



## Gambar 4. Build image berhasil

```
bash scripts/demo-online.sh build
```

```

meet5CloudService [Codespaces: redesigned space adventure]
@SeedFlora → /workspaces/meet5CloudService (main) $ bash scripts/demo-online.sh build

== Dockerfile (resep) → image (paket aplikasi) ==
[+] Building 22.7s (14/14) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 536B
=> [internal] load metadata for docker.io/library/python:3.12-slim
=> [auth] library/python:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 102B
=> [internal] load build context
=> => transferring context: 1.86kB
=> [builder 1/4] FROM docker.io/library/python:3.12-slim@sha256:05cda9777409a9c3ffddd94a4c476b79f0769a0b4857f0c7ed9226b6800b0d6f
=> => resolve docker.io/library/python:3.12-slim@sha256:05cda9777409a9c3ffddd94a4c476b79f0769a0b4857f0c7ed9226b6800b0d6f
=> => sha256:bbc5a12237474ec0b23b5051260d56d6436a24d70d9cad961adebe7a037ed813 249B / 249B
=> => sha256:6912e23eb44595b44dc6d9882b34f305ad6f6e9667cec70b13e5dbc4f8cd97f4 12.12MB / 12.12MB
=> => sha256:75b6a36c64a1cf3520fc4b4aae4739d5caa8a6a05c27835a920a61784c9c32f8 1.29MB / 1.29MB
=> => sha256:ecc510c1e359bdc007b39802570bb7e4dec7d6ebccf357c3201a74299113e05e 29.84MB / 29.84MB
=> => extracting sha256:ecc510c1e359bdc007b39802570bb7e4dec7d6ebccf357c3201a74299113e05e 1.4s
=> => extracting sha256:75b6a36c64a1cf3520fc4b4aae4739d5caa8a6a05c27835a920a61784c9c32f8 0.1s
=> => extracting sha256:6912e23eb44595b44dc6d9882b34f305ad6f6e9667cec70b13e5dbc4f8cd97f4 0.5s
=> => extracting sha256:bbc5a12237474ec0b23b5051260d56d6436a24d70d9cad961adebe7a037ed813 0.5s
=> [runtime 2/5] RUN useradd --create-home --uid 10001 appuser
=> [builder 2/4] WORKDIR /build
=> [builder 3/4] COPY requirements.txt .
=> [builder 4/4] RUN python -m venv /opt/venv && /opt/venv/bin/pip install --no-cache-dir -r requirements.txt
=> [runtime 3/5] WORKDIR /app
=> [runtime 4/5] COPY --from=builder /opt/venv /opt/venv
=> [runtime 5/5] COPY app ./app
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:2eb049694c55831fda3062e5af1668113c20b91207e6d164acd325535eded697
=> => exporting config sha256:c4bcbe1e151f0d15f44bb6de392790d07ac0ee20935256eda4284651610ece72
=> => exporting attestation manifest sha256:0b066f714101b2372dcaa6b9be99e2ca1cc0aaca9e02629848bc8596ded26fb
=> => exporting manifest list sha256:3052ee0a99872f66da33df1f6dc4301dc60a9d7ab1f137a3d0e9c2e75cd3d2cd
=> => naming to docker.io/library/cloud-notes-api:theory-online
=> => unpacking to docker.io/library/cloud-notes-api:theory-online
@SeedFlora → /workspaces/meet5CloudService (main) $
  
```

FINISHED dan nama cloud-notes-api:theory-online menunjukkan image dibangun. Build belum menjalankan server.



Script menjalankan padanan berikut:

```
docker build --tag cloud-notes-api:theory-online .
```

--tag memberi nama image; titik . merupakan build context. Pada pengulangan build, sebagian langkah dapat menampilkan **CACHED**. Bandingkan “resep dibaca” dengan “paket sudah tersedia”; build belum membuka web aplikasi.

```
docker image ls cloud-notes-api
```

Ukuran image bergantung pada base image, dependensi, platform, dan implementasi; bandingkan angka yang benar-benar muncul pada lingkungan kelas.

## 4. Jalankan container dan lihat status Docker

## Gambar 5. Cache build dan container siap

```
bash scripts/demo-online.sh start
```

```

@SeedFlora →/workspaces/meet5CloudService (main) $ bash scripts/demo-online.sh start
=> CACHED [runtime 3/5] WORKDIR /app
0.0sacking to docker.io/library/cloud-notes-api:theory-online
=> CACHED [builder 2/4] WORKDIR /build
0.0s
=> CACHED [builder 3/4] COPY requirements.txt .
0.0s
=> CACHED [builder 4/4] RUN python -m venv /opt/venv && /opt/venv/bin/pip install --no-cache-dir -r requirements.txt
0.0s
=> CACHED [runtime 4/5] COPY --from=builder /opt/venv /opt/venv
0.0s
=> CACHED [runtime 5/5] COPY app ./app
0.0s
=> exporting to image
0.1s
=> => exporting layers
0.0s
=> => exporting manifest sha256:2eb049694c55831fda3062e5af1668113c20b91207e6d164acd325535eded697
0.0s
=> => exporting config sha256:c4bcbe1e151f0d15f44bb6de392790d07ac0ee20935256eda4284651610ece72
0.0s
=> => exporting attestation manifest sha256:474cee70ec29c329c4aa35ba22fc47b087d7cc24007c2e7f35bcd6678a661f05
0.0s
=> => exporting manifest list sha256:f3e2d79b59772e0f10033a20c545b084777196716d93f17382cee53f6825431c
0.0s
=> => naming to docker.io/library/cloud-notes-api:theory-online
0.0s
=> => unpacking to docker.io/library/cloud-notes-api:theory-online
0.0s

== Image → container berjalan; port mesin cloud → port aplikasi ==
fa8e631c2b1bad09ff5ebc68c4db02735e59a622921e4afd1efc61e3a5e1564f
API siap: http://127.0.0.1:8000/health
Swagger: http://127.0.0.1:8000/docs
Di browser Codespaces gunakan tab Ports → port 8000 → Open in Browser → /docs.
o @SeedFlora →/workspaces/meet5CloudService (main) $
o @SeedFlora →/workspaces/meet5CloudService (main) $

```

CACHED menunjukkan input build yang dapat digunakan ulang. API siap setelah health merespons; port host dan container sama-sama 8000 pada demo.

Jika container belum ada, script membuild image lalu menjalankan:

```
docker run --detach --name cloudlab-theory-online \  
  --label edu.binus.cloud-services.demo=week05-online \  
  --publish 8000:8000 \  
  --env APP_ENV=classroom \  
  cloud-notes-api:theory-online
```

| Bagian command                       | Fungsi                                                                                                                          |
|--------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| <code>--detach</code>                | Proses tetap berjalan di latar belakang; terminal kembali menerima perintah.                                                    |
| <code>--name</code>                  | Nama mudah dibaca untuk log, pemeriksaan, dan cleanup.                                                                          |
| <code>--label</code>                 | Penanda kepemilikan demo, agar script menolak mengubah container lain yang kebetulan bernama sama. Bukan mekanisme autentikasi. |
| <code>--publish 8000:8000</code>     | Hubungkan port 8000 mesin cloud dengan port 8000 container.                                                                     |
| <code>--env APP_ENV=classroom</code> | Konfigurasi runtime tanpa mengubah image. Jangan gunakan pola ini untuk menampilkan secret saat presentasi.                     |
| Nama image                           | Paket aplikasi yang dipakai untuk membuat container.                                                                            |

## Gambar 6. Status Up, port, UID, dan log

```
bash scripts/demo-online.sh status
```

```

@SeedFlora → /workspaces/meet5CloudService (main) $ bash scripts/demo-online.sh status

== Status container dan image ==
NAMES                IMAGE                STATUS                PORTS
cloudlab-theory-online  cloud-notes-api:theory-online  Up 38 seconds  0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp

User proses aplikasi (bukan root):
uid=10001(appuser) gid=10001(appuser) groups=10001(appuser)

Log terbaru:
INFO:      Started server process [1]
INFO:      Waiting for application startup.
INFO:      Application startup complete.
INFO:      Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO:      172.17.0.1:56594 - "GET /health HTTP/1.1" 200 OK
@SeedFlora → /workspaces/meet5CloudService (main) $

```

Up menunjukkan proses berjalan. Mapping 8000 menuju port aplikasi; UID 10001 adalah user bukan root. Log Uvicorn menunjukkan server listen 0.0.0.0:8000.

**Yang diamati:** nama container, image, status **Up**, mapping port, user runtime **uid=10001**, dan log Uvicorn. Daemon Docker dan proses aplikasi berjalan di cloud, bukan di laptop penonton.

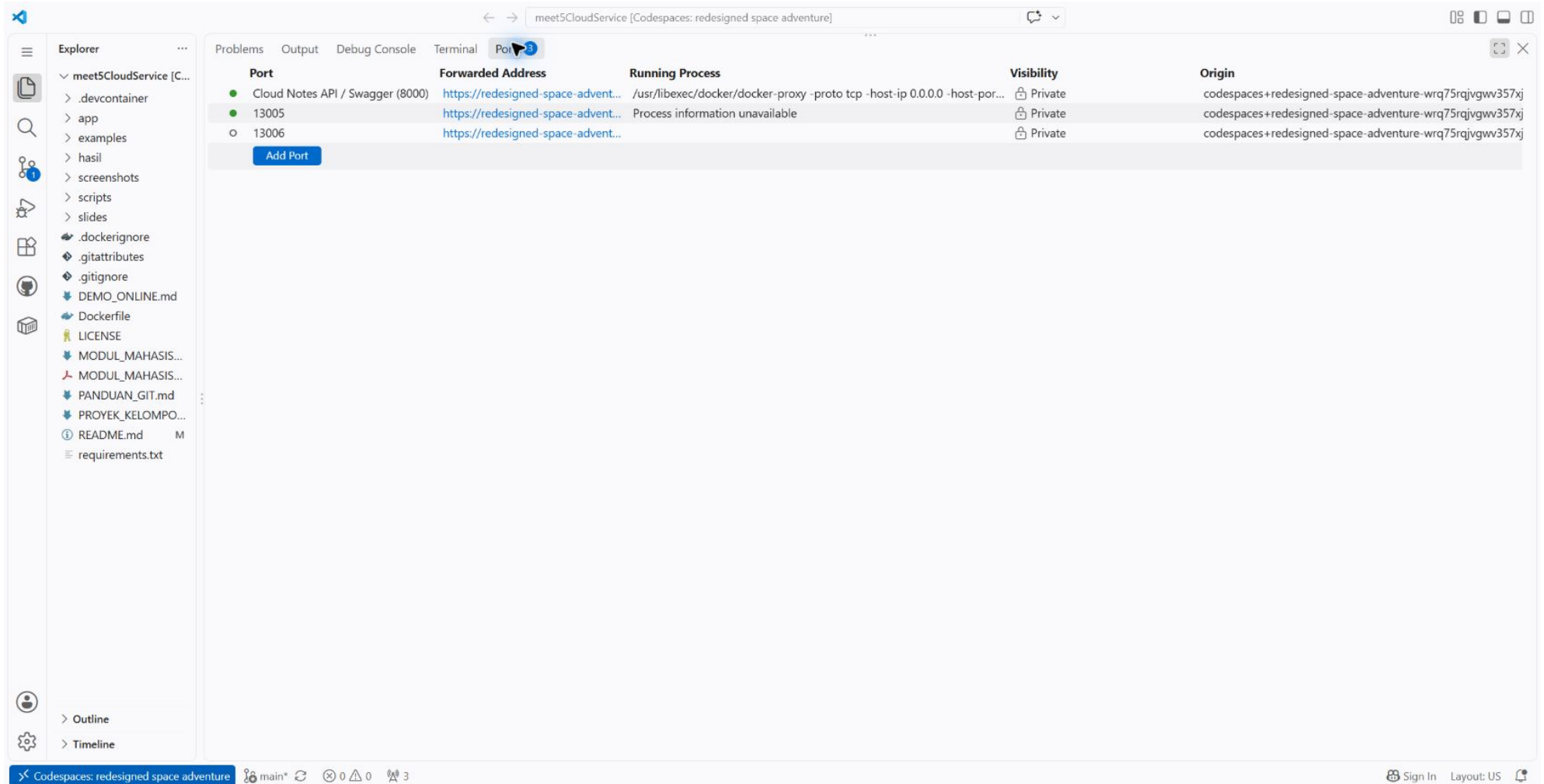
Script tidak memakai **docker system prune**, tidak menghapus image lain, dan menolak container yang label kepemilikannya tidak sesuai. Memanggil **start** lagi menggunakan ulang container demo yang sudah ada. Setelah kode berubah, jalankan **stop** lalu **start** agar container baru memakai image terbaru.

## 5. Tampilkan web/Swagger di layar kelas

1. Buka tab **Ports** pada panel bawah editor Codespaces.
2. Temukan port **8000** dengan label **Cloud Notes API / Swagger**. Jika belum terlihat, pilih **Add Port / Forward a Port**, masukkan **8000**.
3. Pertahankan **Port Visibility: Private**. Dosen yang login dapat membuka aplikasi; presentasi cukup membagikan layar dosen.
4. Klik ikon globe atau **Open in Browser**. GitHub membuat URL forwarding HTTPS.
5. Tambahkan **/docs** pada URL tersebut. Halaman Swagger menampilkan daftar endpoint API.

## Gambar 7. Forwarded port 8000 dengan akses Private

Ports → 8000 → Open in Browser → /docs



Forwarding GitHub menambahkan URL browser ke port host cloud; berbeda dari mapping -p milik Docker.

## Gambar 8. GET health lewat browser menghasilkan HTTP 200

GET /health → Try it out → Execute

### Cloud Notes 0.5 OAS 3.1

/openapi.json

#### default

**GET** /health Health

Parameters

No parameters

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'https://redesigned-space-adventure-wrq75rqjvgwv357xj-8000.app.github.dev/health' \
  -H 'accept: */*'
```

Request URL

```
https://redesigned-space-adventure-wrq75rqjvgwv357xj-8000.app.github.dev/health
```

Server response

| Code | Details                                                                                  |
|------|------------------------------------------------------------------------------------------|
| 200  | <div>Response body</div> <pre>{   "status": "ok",   "service": "cloud-notes-api" }</pre> |

Download

Padanan di terminal Codespaces: `curl -i http://127.0.0.1:8000/health`. status ok menunjukkan liveness API, belum koneksi database.



**Jangan mengetik `http://localhost:8000` pada address bar laptop:** alamat itu menunjuk ke laptop. Gunakan URL pada tab Ports. `localhost` pada terminal Codespaces menunjuk ke mesin cloud, sehingga `curl http://127.0.0.1:8000/health` memang benar di terminal tersebut. Aplikasi di dalam container melayani HTTP; URL HTTPS forwarding disediakan oleh layanan GitHub.

Urutan demonstrasi di Swagger:

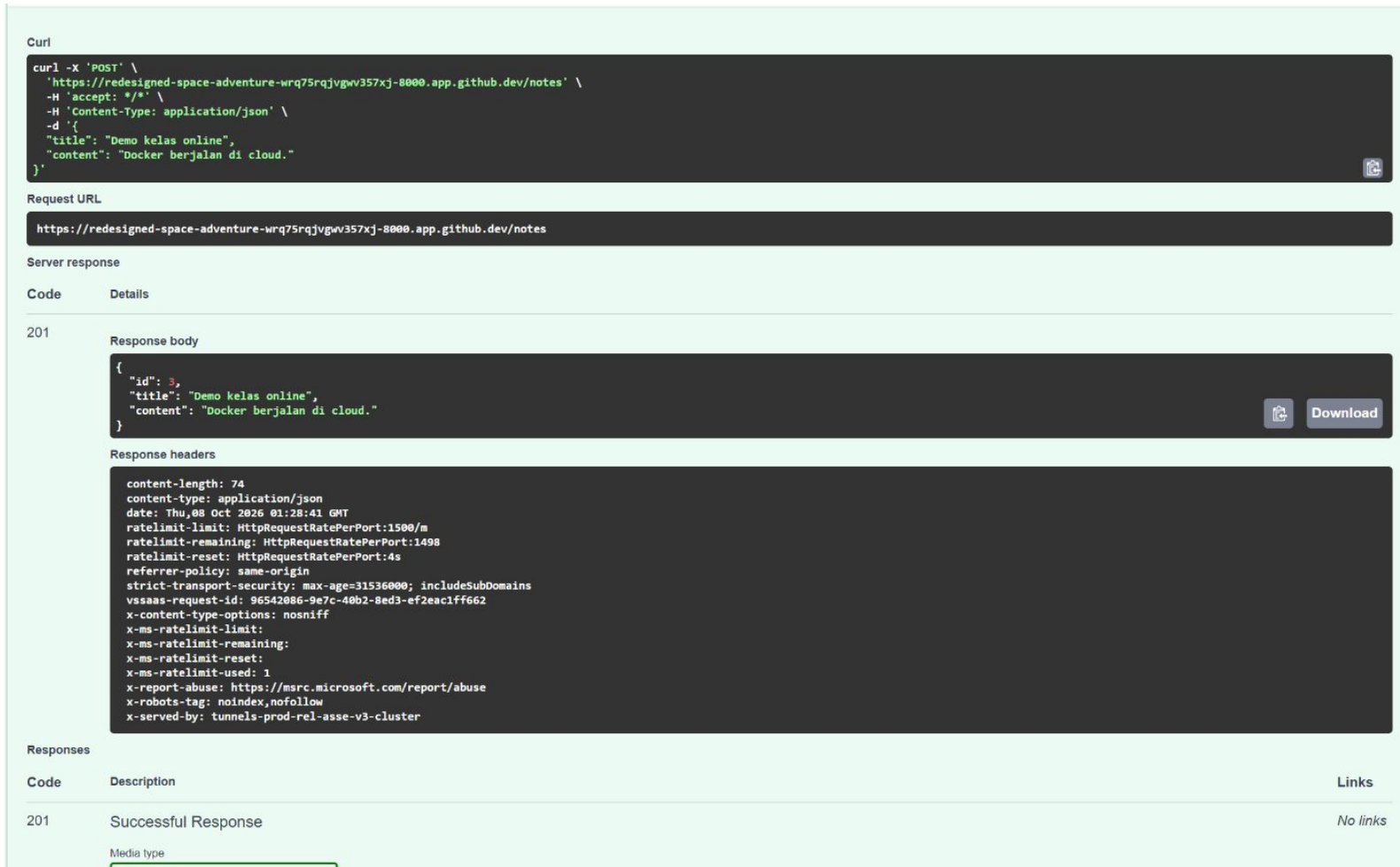
1. **GET /health** → **Try it out** → **Execute**. Respons HTTP 200 menunjukkan aplikasi melayani permintaan.
2. **GET /runtime** → **Execute**. Respons menunjukkan `environment: classroom` dan `storage: memory`; kaitkan dengan `--env` pada `docker run`.
3. **POST /notes** → **Try it out**, masukkan contoh di bawah, lalu **Execute**. Respons HTTP 201 membuat catatan dan menghasilkan `id`.

```
{
  "title": "Demo Docker teori",
  "content": "Aplikasi yang sama berjalan pada mesin cloud."
}
```

4. **GET /notes** → **Execute**. Catatan yang baru dibuat muncul dalam daftar.
5. **POST /notes** dengan `title` berisi spasi saja → HTTP 422. API memvalidasi input; container hidup bukan berarti semua request pasti valid.
6. **GET /notes/{note\_id}** dengan `note_id 0` → HTTP 404. ID 0 tidak dibuat oleh aplikasi ini.

## Gambar 9. POST catatan berhasil dengan HTTP 201

POST /notes → Try it out → isi JSON → Execute



**Curl**

```
curl -X 'POST' \
  'https://redesigned-space-adventure-wrq75rqjvgwv357xj-8000.app.github.dev/notes' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "title": "Demo kelas online",
    "content": "Docker berjalan di cloud."
  }'
```

**Request URL**

https://redesigned-space-adventure-wrq75rqjvgwv357xj-8000.app.github.dev/notes

**Server response**

| Code | Details                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 201  | <p><b>Response body</b></p> <pre>{   "id": 3,   "title": "Demo kelas online",   "content": "Docker berjalan di cloud." }</pre> <p><b>Response headers</b></p> <pre>content-length: 74 content-type: application/json date: Thu, 08 Oct 2026 01:28:41 GMT ratelimit-limit: HttpRequestRatePerPort:1500/m ratelimit-remaining: HttpRequestRatePerPort:1498 ratelimit-reset: HttpRequestRatePerPort:4s referrer-policy: same-origin strict-transport-security: max-age=31536000; includeSubDomains vssaaas-request-id: 96542086-9e7c-40b2-8ed3-ef2eac1ff662 x-content-type-options: nosniff x-ms-ratelimit-limit: x-ms-ratelimit-remaining: x-ms-ratelimit-reset: x-ms-ratelimit-used: 1 x-report-abuse: https://msrc.microsoft.com/report/abuse x-robots-tag: noindex,nofollow x-served-by: tunnels-prod-rel-asse-v3-cluster</pre> |

**Responses**

| Code | Description         | Links    |
|------|---------------------|----------|
| 201  | Successful Response | No links |

Media type

Payload screenshot adalah title Demo kelas online dan content Docker berjalan di cloud. Curl di screenshot memperlihatkan method, Content-Type, dan body. ID 3 adalah hasil uji ini; pada sesi baru ID dapat berbeda.

## Gambar 10. GET daftar membaca catatan yang dibuat

GET /notes → Try it out → Execute

The screenshot displays a REST client interface with the following sections:

- Responses:** The top section, currently empty.
- Curl:** A text area containing the command:

```
curl -X 'GET' \
'https://redesigned-space-adventure-wrq75rqjvgwv357xj-8000.app.github.dev/notes' \
-H 'accept: */*'
```
- Request URL:** A text area containing the URL:

```
https://redesigned-space-adventure-wrq75rqjvgwv357xj-8000.app.github.dev/notes
```
- Server response:** A section showing the response details.
  - Code:** 200
  - Details:** A tab showing the response body and headers.
- Response body:** A JSON array of three note objects:

```
[
  {
    "id": 1,
    "title": "Demo Docker teori",
    "content": "Resep + image + container; data sementara di memori."
  },
  {
    "id": 2,
    "title": "Demo Docker teori",
    "content": "Resep + image + container; data sementara di memori."
  },
  {
    "id": 3,
    "title": "Demo kelas online",
    "content": "Docker berjalan di cloud."
  }
]
```
- Response headers:** A list of HTTP headers:

```
content-length: 288
content-type: application/json
date: Thu, 08 Oct 2026 01:29:10 GMT
ratelimit-limit: HttpRequestRatePerPort:1500/m,ClientConnectionsPerPort
ratelimit-remaining: HttpRequestRatePerPort:1499
ratelimit-reset: HttpRequestRatePerPort:44s
referrer-policy: same-origin
strict-transport-security: max-age=31536000; includeSubDomains
vssas-request-id: bc3485a3-6cd4-4f08-8173-ecb0967d1dff
x-content-type-options: nosniff
x-ms-ratelimit-limit:
x-ms-ratelimit-remaining:
x-ms-ratelimit-reset:
x-ms-ratelimit-used: 1
x-report-abuse: https://msrc.microsoft.com/report/abuse
x-robots-tag: noindex,nofollow
x-served-by: tunnels-prod-rel-asse-v3-cluster
```

Padanan terminal: `curl -i http://127.0.0.1:8000/notes`. Daftar memuat catatan dari script test dan POST browser. HTTP 200 tidak membuktikan data persisten.

Screenshot utama yang berguna untuk catatan kelas: terminal build, status container + port, tab Ports, Swagger `/health`, hasil POST 201, daftar catatan, dan daftar kosong setelah restart. Tampilkan command yang menghasilkan setiap screenshot dan jelaskan outputnya.

## 6. Pemeriksaan end to end dan data sementara

Untuk memperlihatkan pemeriksaan otomatis yang menguji aplikasi sungguhan:

## Gambar 11. Pemeriksaan end to end lulus

```
bash scripts/demo-online.sh test
```

```

meet5CloudService [Codespaces: redesigned space adventure]
Problems Output Debug Console Terminal Ports
@SeedFlora → /workspaces/meet5CloudService (main) $ bash scripts/demo-online.sh test

== HTTP: layanan hidup, konfigurasi runtime, validasi, data, dokumentasi ==
PASS GET /health → HTTP 200
{"status":"ok","service":"cloud-notes-api"}
PASS GET /runtime → HTTP 200
{"environment":"classroom","storage":"memory"}
PASS POST /notes → HTTP 201
{"id":2,"title":"Demo Docker teori","content":"Resep → image → container; data sementara di memori."}
PASS GET /notes/2 → HTTP 200
{"id":2,"title":"Demo Docker teori","content":"Resep → image → container; data sementara di memori."}
PASS GET /notes → HTTP 200
[{"id":1,"title":"Demo Docker teori","content":"Resep → image → container; data sementara di memori."},{
  "id":2,"title":"Demo Docker teori","content":"Resep
  → image → container; data sementara di memori."}]
PASS POST /notes → HTTP 422
{"detail":[{"type":"value_error","loc":["body","title"],"msg":"Value error, teks tidak boleh hanya berisi spasi","input":"  ",
  "ctx":{"error":{}}]}]
PASS GET /notes/0 → HTTP 404
{"detail":"Note tidak ditemukan"}
PASS GET /docs → HTTP 200

Semua pemeriksaan demo lulus. Ini demonstrasi formatif, tanpa tugas atau nilai lab terpisah.
@SeedFlora → /workspaces/meet5CloudService (main) $

```

200 untuk baca/health/docs; 201 untuk create; 422 untuk judul kosong; 404 untuk ID 0. PASS mengikuti respons layanan sungguhan.

Script memeriksa respons `/health`, nilai `APP_ENV`, membuat catatan, membaca catatan tersebut, memeriksa daftar, menolak input kosong dengan HTTP 422, memeriksa HTTP 404, dan membuka `/docs`. **PASS** diberikan setelah HTTP status dan isi penting sesuai; halaman Swagger sendiri tidak dicetak seluruhnya ke terminal.

Demonstrasi sifat in-memory:

## Gambar 12. Restart mengosongkan data memori

```
bash scripts/demo-online.sh test --restart
```

```

meet5CloudService [Codespaces: redesigned space adventure]
Problems Output Debug Console Terminal Ports
@SeedFlora → /workspaces/meet5CloudService (main) $ bash scripts/demo-online.sh test --restart

== HTTP: layanan hidup, konfigurasi runtime, validasi, data, dokumentasi ==
PASS GET /health → HTTP 200
{"status":"ok","service":"cloud-notes-api"}
PASS GET /runtime → HTTP 200
{"environment":"classroom","storage":"memory"}
PASS POST /notes → HTTP 201
{"id":4,"title":"Demo Docker teori","content":"Resep → image → container; data sementara di memori."}
PASS GET /notes/4 → HTTP 200
{"id":4,"title":"Demo Docker teori","content":"Resep → image → container; data sementara di memori."}
PASS GET /notes → HTTP 200
[{"id":1,"title":"Demo Docker teori","content":"Resep → image → container; data sementara di memori."},{
{"id":2,"title":"Demo Docker teori","content":"Resep → image → container; data sementara di memori."},
{"id":3,"title":"Demo kelas online","content":"Docker berjalan di cloud."},{
{"id":4,"title":"Demo Docker t
eori","content":"Resep → image → container; data sementara di memori."}]
PASS POST /notes → HTTP 422
{"detail":[{"type":"value_error","loc":["body","title"],"msg":"Value error, teks tidak boleh hanya berisi spasi","input":"  ","ctx":{"error":{}}}]
PASS GET /notes/0 → HTTP 404
{"detail":"Note tidak ditemukan"}
PASS GET /docs → HTTP 200

== Restart proses: catatan in-memory menghilang (bukan penyimpanan database) ==
cloudlab-theory-online
PASS GET /notes → HTTP 200
[]
PASS: data sementara hilang sesudah restart. Volume saja tidak mengubah dict Python menjadi database.

Semua pemeriksaan demo lulus. Ini demonstrasi formatif, tanpa tugas atau nilai lab terpisah.
@SeedFlora → /workspaces/meet5CloudService (main) $

```

Setelah docker restart, GET /notes menghasilkan []. Image masih ada. Memori proses tidak menjadi persisten hanya karena diberi volume.

Command ini membuat catatan uji, me-restart **container demo**, lalu memastikan GET [/notes](#) menghasilkan `[]`. Data hilang karena dict Python disimpan dalam memori proses. Restart proses saja sudah menghapusnya; menambahkan volume tidak otomatis membuat dict menjadi database. Database dan persistensi akan dikembangkan pada tahapan proyek berikutnya.

Ini pertanyaan diskusi formatif, beserta jawabannya: “Apakah image terhapus saat container restart?” **Tidak**. Image masih ada. Yang hilang pada contoh ini adalah data di memori proses aplikasi.

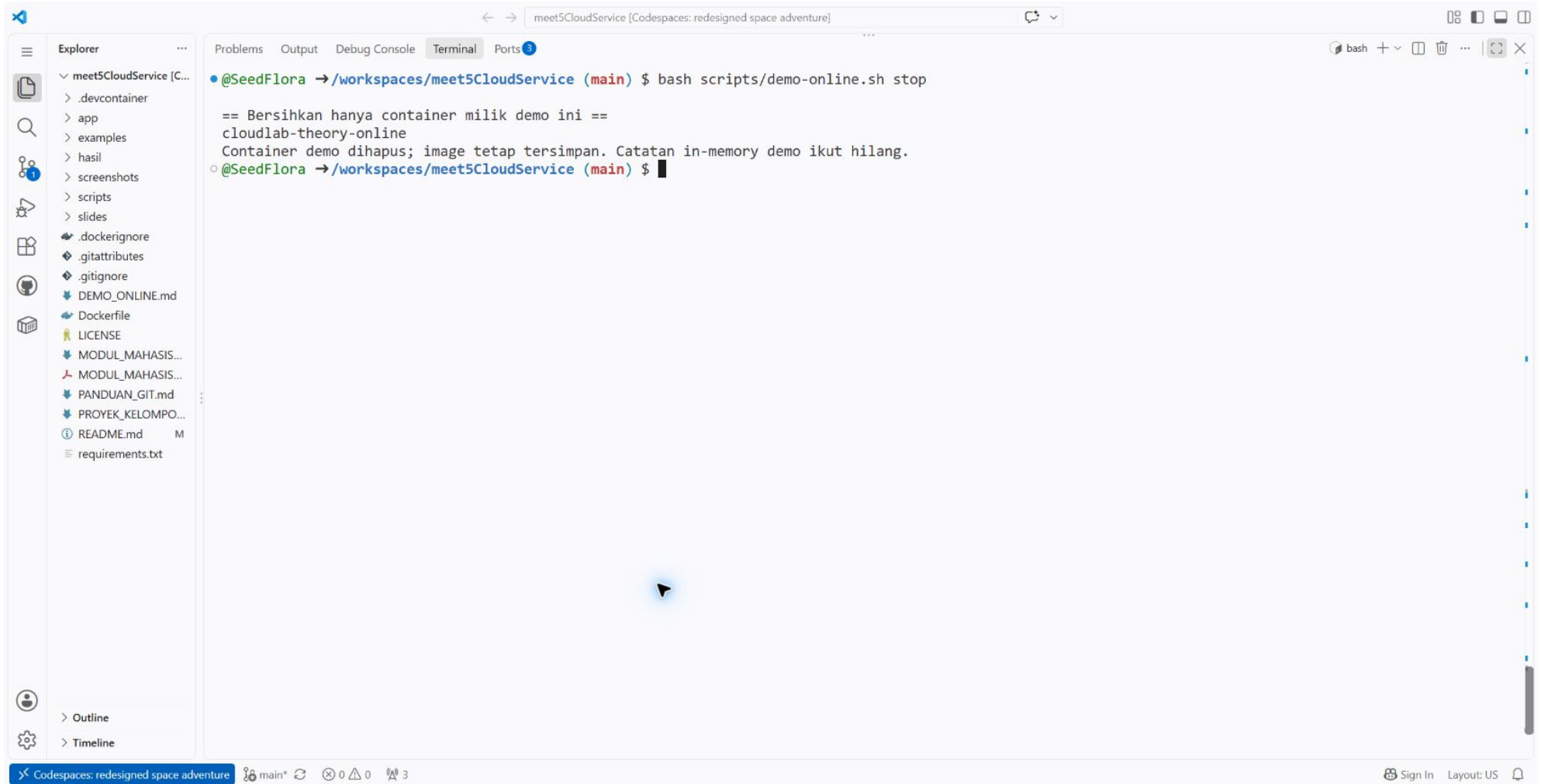
## 7. Berhenti setelah kelas

Hapus container demo sendiri:



## Gambar 13. Container demo dibersihkan

```
bash scripts/demo-online.sh stop
```



Menghapus hanya container bernama dan berlabel milik demo. Image tetap tersedia; container lain tidak dibersihkan.

Image tetap tersedia sehingga build berikutnya dapat memakai cache. Semua catatan demo sementara hilang. Script tidak mengubah container lain.

Selanjutnya hentikan **Codespaces**, bukan hanya menutup tab: buka [github.com/codespaces](https://github.com/codespaces), pilih menu ... di Codespace kelas → **Stop codespace**. Menghentikan Codespace menghentikan penggunaan compute; penyimpanan tetap dihitung selama Codespace masih ada.

## Gambar 14. Codespace sudah Stopped

github.com/codespaces → menu ... → Stop codespace

The screenshot shows the GitHub Codespaces web interface. At the top, a dark blue banner displays the navigation path: `github.com/codespaces` → menu ... → Stop codespace. Below this, a light blue notification bar states: "Codespace 'redesigned space adventure' stopped." with a close button (X). The main interface is divided into a left sidebar and a main content area. The sidebar includes a 'Codespaces' header, a search bar, and a list of repositories under 'By repository': SeedFlora/linux-fundamental-2, SeedFlora/linux-fundamentals, SeedFlora/meet3CloudService, and SeedFlora/meet5CloudService. The main content area is titled 'Your codespaces' and features a 'Go to docs' button and a green 'New codespace' button. Below this, there's a section 'Explore quick start templates' with four options: Blank, React, Jupyter Notebook, and .NET. Each template has a 'Use this template' button. At the bottom, a section 'Owned by SeedFlora' lists three codespaces: 'redesigned space adventure' (last used 14 minutes ago), 'studious barnacle' (last used 7 days ago), and 'super doodle' (last used 21 days ago). Each entry shows the repository name, the current branch (main\*), and the last used time.

Codespace "redesigned space adventure" stopped.

**Your codespaces** [Go to docs](#) [New codespace](#)

**Explore quick start templates** [See all](#)

- Blank** By github ⓘ  
Start with a blank canvas or import any packages you need.  
[Use this template](#)
- React** By github ⓘ  
A popular JavaScript library for building user interfaces based on UI components.  
[Use this template](#)
- Jupyter Notebook** By github ⓘ  
JupyterLab is the latest web-based interactive development environment for notebooks, code, and data.  
[Use this template](#)
- .NET** By github ⓘ  
A full-stack web application template written in C#.  
[Use this template](#)

**Owned by SeedFlora**

| Repository                    | Codespace Name             | Branch | Status                                 | Specs                             | Last Used                |
|-------------------------------|----------------------------|--------|----------------------------------------|-----------------------------------|--------------------------|
| SeedFlora/meet5CloudService   | redesigned space adventure | main*  | This codespace has uncommitted changes | 2-core • 8GB RAM • 32GB • 1.51 GB | Last used 14 minutes ago |
| SeedFlora/meet3CloudService   | studious barnacle          | main*  | This codespace has uncommitted changes | 2-core • 8GB RAM • 32GB • 2.2 GB  | Last used 7 days ago     |
| SeedFlora/linux-fundamental-2 | super doodle               | main*  | This codespace has uncommitted changes | 2-core • 8GB RAM • 32GB • 7.06 GB | Last used 21 days ago    |

Stopped menghentikan compute. Lingkungan disimpan untuk dibuka kembali, sehingga storage masih dihitung. Menutup tab saja bukan langkah penghentian.

Pada **8 Oktober 2026**, akun personal GitHub Free mendapat **120 core-hours compute** dan **15 GB-month storage** per bulan. Mesin 2 core mengonsumsi 2 core-hours untuk satu jam aktif, sehingga 120 core-hours setara sekitar 60 jam aktif pada mesin itu. Kuota bersama penggunaan Codespaces lain pada akun; bukan jaminan gratis tanpa batas. Detail dan kebijakan terbaru: [GitHub Codespaces billing](#).

Jika Codespace akan **dihapus**, commit dan push perubahan yang ingin disimpan terlebih dahulu. Penghapusan membuang perubahan yang belum tersimpan di remote. Untuk demo tanpa perubahan kode, cukup Stop setelah kelas; hapus lingkungan jika sudah tidak diperlukan.

## 8. Jalur cadangan: Killercoda Docker Playground

Gunakan [Killercoda Docker Playground](#) jika Codespaces tidak tersedia. Login sesuai permintaan layanan, tunggu terminal siap, lalu jalankan:

```
git clone https://github.com/SeedFlora/meet5CloudService.git
```

```
cd meet5CloudService
```

```
bash scripts/demo-online.sh check
```

```
bash scripts/demo-online.sh start
```

```
bash scripts/demo-online.sh test
```

Untuk web, buka panel **Traffic / Port Access** pada antarmuka playground, masukkan port **8000**, lalu tambahkan [/docs](#) pada URL yang diberikan. Nama menu dapat berbeda menurut antarmuka layanan. Untuk demonstrasi terminal saja, [curl](#) dan script tetap dapat ditampilkan tanpa membuka akses web. URL akses Killercoda bukan port private berautentikasi seperti Codespaces; gunakan hanya data dummy demo.

Sesi gratis Killercoda dibatasi sekitar **1 jam per lingkungan** dan bersifat sementara; setelah sesi berakhir lingkungan dihapus. Jangan menaruh data kelas penting atau secret produksi di dalamnya. Simpan perubahan ke repo milik sendiri sebelum lingkungan hilang. Rincian: [FAQ resmi Killercoda](#).

**Play with Docker tidak digunakan:** [pengumuman pada repo resminya](#) menyatakan layanan tidak tersedia mulai 1 Maret 2026. Gunakan Codespaces atau playground yang masih tersedia.

## 9. Kendala umum

| Gejala                                                     | Langkah                                                                                                                                                                                                                |
|------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Tidak ada terminal Linux / Docker di editor GitHub         | Pastikan membuka <b>Codespaces</b> , bukan editor ringan <code>github.dev</code> .                                                                                                                                     |
| Docker daemon belum siap                                   | Tunggu proses konfigurasi, jalankan <code>docker version</code> dan <code>check</code> lagi. Jika konfigurasi repo baru ditambahkan pada Codespace lama, <b>Codespaces: Rebuild Container</b> ; simpan perubahan dulu. |
| Kuota habis / pembuatan Codespace ditolak                  | Periksa kuota akun dan gunakan Killercoda sebagai cadangan.                                                                                                                                                            |
| Port 8000 sudah dipakai                                    | Jalankan <code>DEMO_PORT=18005 bash scripts/demo-online.sh start</code> ; gunakan awalan yang sama untuk <code>test</code> dan <code>status</code> . Forward port 18005 secara manual pada tab Ports.                  |
| Script menolak nama container                              | Ada container lain dengan nama demo tetapi label berbeda. Jangan menghapusnya tanpa memeriksa pemiliknya. Gunakan Codespace baru untuk kelas.                                                                          |
| Browser menampilkan 404 di /                               | Aplikasi ini tidak membuat homepage. Buka <code>/docs</code> , <code>/health</code> , atau <code>/runtime</code> .                                                                                                     |
| Gagal mengakses <code>localhost</code> dari browser laptop | Buka URL HTTPS pada tab <b>Ports</b> .                                                                                                                                                                                 |
| Kode diubah tetapi respons lama                            | <code>stop</code> , lalu <code>start</code> ; container lama tidak otomatis diganti ketika image dibuild ulang.                                                                                                        |
| Catatan hilang sesudah restart                             | Perilaku yang diharapkan: penyimpanan masih di memori proses.                                                                                                                                                          |

## 10. Sumber resmi

- [GitHub Codespaces quickstart](#): pembuatan lingkungan melalui browser.
- [Port forwarding di Codespaces](#): port dan visibility.
- [Docker-in-Docker Feature](#): Docker pada dev container.
- [Referensi Dockerfile](#): instruksi dan perilakunya.
- [Multi-stage builds](#): pemisahan builder dan runtime.
- [Build secrets](#): gunakan secret mounts untuk kebutuhan build, bukan ARG/ENV untuk secret.
- [Codespaces billing](#) dan [Killercoda FAQ](#): batas penggunaan.